

When multiple developers work on a single remote repository, you cannot modify the order of commits in the remote repository. Hence, you should try to formulate your local commits on top of the remote repository commits. So use rebase to put your local commits on top of remote repository commits and push the changes. To add your local commits on top of remote repo commits, use the following command:

```
$ git fetch
$ git rebase -i origin/master
```

This command will interactively modify the local commits (master branch) on top of commits from the origin of the remote repo's master branch.

Tracking the remote branch on a local branch

It is possible to create a local branch that tracks a remote branch with the following command:

```
git checkout -b localbranch remote_repo/newbranch
```

Fetching changes

The *git fetch* command is used to download commits from remote repositories, but it does not attempt to merge the code. An optional parameter is the repository name, as in *git fetch repo_name*.

Stashing changes

Stash is a very interesting feature. When you make local changes, you may need to switch to some other branch to check something and come back, but may not want to lose your local changes that are not yet committed. The *git stash* command preserves the current repository state for the current branch (with every local change) in its memory. The Git stash memory is a stack -- a LIFO list. To get the stashed state back, you can first use *git stash list* to view the contents of the stash memory; to apply the last stashed item back, use *git stash apply*.

Displaying objects

The *git show* command is used to display any object in the Git repo—commits, blobs, the tree, etc.

Tagging

Marking release versions of code helps you track code with respect to features or based on different criteria. Git has the built-in capability to tag different commits with tag descriptions. To list available tags, use *git tag -l*. To tag the current *HEAD*, use the following command:

```
$ git tag -a 'tagname' -m 'tag message' HEAD
```

To tag a particular commit, use *COMMIT_ID* instead of *HEAD*. To display details about a tag, just use *git show*

tagname. To push tags to a remote repo, use *git push --tags*. To delete a tag from local and remote repos, use the command given below:

```
$ git tag -d tag_name // delete local tag
$ git push origin :tag_name // delete tag from remote repository
```

Creating and using patches

Commits in Git can be represented as a series of diffs or patches applied one after the other. You can create patches from commits, email them, and the recipients can apply them to their repository. To create a patch from *COMMIT_ID* to the current *HEAD*, use:

```
$ git format-patch COMMIT_ID
```

This will create a series of *.patch* files. To apply and create commits on a repo from these *.patch* files, use *git am patch_file.patch*. To apply the patch's changes to only modify files locally, but not create the commits, use *git apply patch_file.patch*.

Cherry-picking changes

Using Git, it is possible to pick an individual commit from any branch in the current repo, and try to apply it on top of the current branch *HEAD*. This feature is known as cherry-picking. To cherry-pick a commit, use: *git cherry-pick COMMIT_ID*.

Creating archives from the repository

It is possible to create a source-code archive from the Git repository. To create *file.tar* with all the latest code in the *HEAD*, run *git archive -o file.tar HEAD*.

Cleaning a repository

While working with code, you create lot of temporary files in the working directory. At some point, you may need to clean out all the files except those tracked by the Git repository. For this, use the *git clean* command.

Migrating an SVN repository to Git

Migrating all code from a Subversion repository without losing any history is super easy with the following steps. First, create a *users.txt* file in the following format, listing the Subversion users and Git username:

```
svn_user1 = Git User1 <email>
svn_user2 = Git User2 <email>
```

Next, initialise the Git repo and do a fetch, as shown below:

```
$ git svn init svn_repo_url
$ git config svn.authorsfile users.txt
$ git svn fetch
```